1. Aiken

Program Name: Aiken.java **Input File:** None

Using the term "bug" to refer to a design defect has long been an accepted definition of the word since the early 1800s when Thomas Edison used the phrase to describe a problem with his telephone designs. That said, on September 9, 1947, the very first computer bug was discovered; however, this was not your typical "software bug" as it was a real-life moth that was causing issues with the computer's hardware! Older computers, like that of the Mark II Aiken, used to use punch cards to write programs. However, as Grace Hopper would find out, these cards had an unintentional weakness – actual living bugs. To help commemorate the etymology of this term, print a simple ASCII art of the moth that Grace Hopper found on that fateful day in 1947.

Input

None

Output

Output the moth as shown below. Note that a "ruler" has been given for your convenience below the moth and should NOT be included in the actual output.

Example Input File

```
none
```

Example Output to Screen

```
 //\          /\\
|| * \ . . / * ||
 \\____\X/____//
  / *  /O\  * \
  \__/  "   \__/
12345678901234567
```

2. Nibble

Program Name: Nibble.java **Input File: nibble.dat**

Like any other science, in the world of Computer Science, we have our own set of standardized units. Two common standardized units important to Computer Science are the bit, which represents a single '0' or '1' (hence, where the name binary comes from), and a byte, which is formed from eight bits. However, there is a lesser-known intermediate unit between the bit and the byte – the nibble. A nibble is formed from four bits; hence, two nibbles make up a byte. Having recently learned about this new unit, you wish to write a program that, given a string, first determines if it is a valid binary string, and then, if it is, prints out the binary string separated into nibbles.

Input

The first line will contain a single integer $1 \leq n \leq 100$ denoting the number of data sets that follow. Each data set consist of a string w which will consist only of ASCII printable characters with no whitespace. It should be noted that $1 \leq |w| \leq 100$.

Output

For each of the n testcases, first determine if the string is a binary string or not. If not, then print the string "`Wrong base...`". If, however, the string is a valid binary string, then separate each nibble of the string by a single space (that is, the character ' '). If the length of the string is not a multiple of four, then append the fewest number of 0's to the end of the bit string to make it a multiple of four, and then separate it into nibbles as described above.

Example Input File

```
8
0100100001000101010011000100110001001111
0101011101101111011100100110110001100100000100001
0101011101101111011100100110110001100100000100003
010101110110111101110010011011000110010000100
0000
1111
4444
0100110110111011001011110001101000010001
```

Example Output to Screen

```
0100 1000 0100 0101 0100 1100 0100 1100 0100 1111
0101 0111 0110 1111 0111 0010 0110 1100 0110 0100 0010 0001
Wrong Base...
0101 0111 0110 1111 0111 0010 0110 1100 0110 0100 0010 0000
0000
1111
Wrong Base...
Wrong Base...
```

3. Boxing

Program Name: Boxing.java **Input File: boxing.dat**

Some languages are read left-to-right, while other languages are read right-to-left. However, these methods bore you and you wish to make a language where words are read in a more creative fashion. Introducing your new language: Boxing. An English sentence in the language of Boxing is formed by concentric square rings of letters. The outermost ring consists entirely of the left-most character in the English sentence. The second-outermost ring consists entirely of the second left-most character in the English sentence. This process is repeated until it reaches the innermost ring, which consists entirely of the right-most character in the English sentence. Because of this, so long as you start reading in the outermost ring and stop reading in the innermost ring, the language of Boxing can be read in multiple directions. These include left-to-right, right-to-left, top-to-bottom, bottom-to-top, and diagonally. Write a program that will translate a sentence in English to a sentence in Boxing.

Input

The first line will contain a single integer $1 \leq n \leq 25$ denoting the number of testcases to follow. Each of the next n lines will consist of a single sentence that may contain any printable ASCII character except for whitespaces, with the exception of the space, which is allowed. For each sentence w, it is guaranteed that $1 \leq |w| \leq 25$ and that each sentence does not have any leading or trailing spaces.

Output

For each testcase, print the translated sentence in Boxing, followed by a single line consisting of five '^'s (that is, the string "^^^^^").

Example Input File

```
3
Apple
Hi!
One Two
```

Example Output to Screen

```
AAAAAAAAA
AppppppppA
AppppppppA
AppllllppA
AppleelppA
AppllllppA
AppppppppA
AppppppppA
AAAAAAAAA
^^^^^
HHHHH
HiiiH
Hi!iH
HiiiH
HHHHH
^^^^^
```

Continues next column...

Continued from previous column…

```
OOOOOOOOOOOOO
OnnnnnnnnnnnO
OneeeeeeeeeenO
One       enO
One TTTTT enO
One TwwwT enO
One TwowT enO
One TwwwT enO
One TTTTT enO
One       enO
OneeeeeeeeeenO
OnnnnnnnnnnnO
OOOOOOOOOOOOO
^^^^^
```

4. Pangram

Program Name: Pangram.java **Input File: pangram.dat**

A pangram is a special type of sentence where, given an alphabet, the sentence contains every letter in the alphabet at least once. Moreover, a perfect pangram is a special type of pangram where every letter in the alphabet exactly once. Given a set of alphabets and sentences, determine whether each sentence is a pangram, a perfect pangram, or neither in its respective language.

Input

The first line will consist of a single integer, $1 \leq n \leq 100$, which denotes the number of testcases to follow. For each of the n testcases, the first line will denote the alphabet, which will be a string of non-repeating, non-whitespace ASCII characters. It should be noted that alphabets are case sensitive. The second line will denote the given sentence to classify. It should be noted that sentences have a length between 1 and 100 characters in length and are allowed to contain characters that are not in their alphabets.

Output

For each of the n testcases, on their own line, either print the string "`perfect pangram`", "`pangram`", or "`neither`" depending on which classification applies to each sentence.

Example Input File

```
5
abcdefghijklmnopqrstuvwxyz
the quick brown fox jumps over the lazy dog
acdeghijkmot
jim got hacked
abcdefghijklmnopqrstuvwxyz
this sentence is not a pangram
235+=
2 + 3 = 5
AabcdefghijklmnOopqrstUuvwxyz
sphinx of blAck qUartz, judge my vOw
```

Example Output to Screen

```
pangram
perfect pangram
neither
perfect pangram
perfect pangram
```

5. Zeros

Program Name: Zeros.java **Input File: zeros.dat**

Learning to work with bit strings can be quite an intimidating task for many up-and-coming computer scientists. Looking to begin your journey of getting comfortable with bit strings, you wish to know how many bit strings of width w there are which don't contain any consecutive 0s. Note: the "empty string" (denoted in theory as ε) is a bit string with width 0 which does not contain any characters.

Input

The first line will consist of a single integer $1 \leq n \leq 10^3$ denoting the number of test cases to follow. The next n lines will each consist of a single integer $0 \leq w_i \leq 10^5$ where w_i is the desired width of the i^{th} test case.

Output

For each of the n test cases, output the number of unique bit strings of width w_i modulo $10^9 + 7$ as this number can get quite large.

Example Input File

```
11
0
1
2
3
10
16
20
40
70
100
200
```

Example Output to Screen

```
1
2
3
5
144
2584
17711
267914296
8390086
470199269
878671356
```

6. Endless

Program Name: Endless.java **Input File: endless.dat**

Tic-Tac-Toe is a popular game which consists of a 3x3 board, where players go back and forth taking turns marking squares on the board with either an 'X' or an 'O' until one player is able to get three of their symbols in a row (or neither player is able to). Some people are appalled by the idea of games ending in a tie; thus, a variant of Tic-Tac-Toe was invented to circumvent just this: Endless Tic-Tac-Toe. This variant plays with the additional rule that a symbol on the board disappears (thus, leaving the spot it occupied blank again) **after** 6 additional symbols have been written after that symbol was initially written. This prevents a tie from ever occurring in a finite amount of time; however, this still does allow for a player to win once three of their own symbols have been marked in a row. Note that winning rows exist on any horizontal, vertical, or diagonal line. Given a set of turns between two players, determine which player will win, if any.

Input

The first line will consist of a single integer $1 \leq n \leq 10^3$ denoting the number of games to follow. For each of the n games, the first line will consist of a single integer $1 \leq m \leq 10^3$, denoting the number of moves that occur during that game. Then, the next m lines consist of two space-separated integers, r_i and c_i, denoting the row and column of the i^{th} move. It should be noted that $1 \leq r_i, c_i \leq 3$ for all i.

Output

Assuming that the player who marks the board with 'X's moves first, either print out the string "`Player using 'X's wins`", "`Playing using 'O's wins`", or "`Neither player has won yet...`". It should also be noted that the number of moves described for a given game may end up being greater than the number of moves that it actually takes for a winner to be determined. In such a case, those extra moves are unimportant and should not be considered.

Example Input File

```
3
18
3 3
1 1
3 1
3 2
2 2
1 3
1 2
3 1
2 3
3 1
3 2
```

Continues next column…

Continued from previous column…

```
1 1
2 1
2 2
3 3
1 3
1 2
3 1
7
2 2
1 2
3 3
1 1
```

Continues next column…

Continued from previous column…

```
1 3
3 1
2 3
8
2 2
3 1
1 3
1 1
2 1
2 3
3 2
2 2
```

Example Output to Screen

```
Player using 'O's wins
Player using 'X's wins
Neither player has won yet...
```

7. Experience

Program Name: Experience.java **Input File: experience.dat**

Many games have different ways of calculating and representing the idea of "experience". In one such popular 3D cubic game about mining and crafting, experience can be rewarded to the player when mobs are slain; in which case, experience "orbs" are dropped, which the player can then collect. However, not all experience orbs are worth the same amount. In this game, the following values can be assigned to a given experience orb:

1, 3, 7, 17, 37, 73, 149, 307, 617, 1237, 2477

However, since the amount of experience dropped for a given mob does not always perfectly equal one of the amounts listed above, the experience instead has to be split up into differently sized orbs which sum to the amount in question. Moreover, as to not waste the number of particles that must be rendered at a given point in time, this calculation is done using the fewest number of orbs possible. Write a program that, given the amount of experience dropped by a given mob, returns the number of orbs of each size that is required to evenly sum up to the amount denoted using the fewest number of orbs possible. If multiple such minimal solutions exist, output the one that uses the greatest number of higher valued experience orbs. That is, ties are broken starting with comparing solutions that use more 2477 orbs, then, if ties exist, those that use more 1237 orbs, and so on.

Input

The first line will consist of a single integer $1 \leq n \leq 2 \cdot 10^4$ denoting the number of distinct mobs to follow. The next n lines will consist of a single integer $1 \leq m_i \leq 10^7 - 1$ denoting the amount experience dropped by the i^{th} mob.

Output

For each of the n experience values, output a string of 11 space-separated integers denoting the number of each respective experience orb value (following the order given above) that is needed to sum to m_i using the least number of total experience orbs.

Example Input File	Example Output to Screen
10	0 1 0 0 0 0 0 0 0 0 0
3	2 1 1 0 0 0 0 0 0 0 0
12	2 1 0 0 0 0 0 0 0 0 0
5	1 0 2 0 0 0 0 0 0 0 0
15	0 1 1 1 0 1 0 0 0 0 0
100	2 1 1 0 1 0 1 1 0 0 0
505	0 1 0 1 0 0 0 0 0 0 0
20	2 0 2 0 0 1 1 0 1 1 4
12000	0 2 0 0 1 0 0 0 0 0 0
43	1 1 1 1 1 0 0 0 0 0 0
65	

8. Ladder

Program Name: Ladder.java **Input File: ladder.dat**

A word ladder is an ordered series of words which maintains the following invariant: two adjacent words differ by a single character. It should be noted that the addition or removal of letters is not allowed – only the swapping of a single character. Given a dictionary of words, determine if it is possible to get from some starting word to an ending word, and if it is possible to, how many unique ways are there to do so using the fewest number of swaps.

Input

The first line of input will consist of two integers $1 \leq n \leq 2252$ denoting the number of words in the dictionary and $1 \leq m \leq 300$ denoting the number of pairs of starting and ending word testcases there are. The next line will consist of n space-separated words which denote the dictionary of words at your disposal. The next m lines will consist of two space-separated words denoting the starting and ending words of the i^{th} testcase, respectively. It should be noted that all words will be of length 4, consist only of uppercase letters, and no duplicate words will exist in the dictionary. Additionally, the start word will not be the same as the ending word.

Output

For each of the m testcases, either print out the string "Not Connected." or print out two space-separated integers, d and u, the shortest distance between the starting word and the ending word and the number of unique ways there are to get from the starting word to the ending word using d edits, respectively.

Example Input File *(indented lines are continuation of previous line)*

```
21 4
COLD TAIL TALL CORM WOLD HEAD HEAL TEAL WARM CORD WORM TELL SHIP
     SHOW SHOP STOW CARD WARD WORD SNAP WALD
SHIP STOW
HEAD TALL
COLD WARM
TAIL SHOP
```

Example Output to Screen

```
4 1
5 1
5 7
Not Connected.
```

9. Ducks

Program Name: Ducks.java **Input File: ducks.dat**

Ever heard the saying "you need to get your ducks in a row?" Well, you've decided to take a very literal interpretation of this problem and wish to put several ducks in some deterministic order. However, seeing how getting live ducks to stand still would be quite an impossible task, you have opted to use rubber ducks instead. Every duck has a height, color, and hat. Since you want to be able to see each duck when looking straight on at the duck at the front of the line, you wish for the ducks to be sorted in ascending order according to height. Moreover, in cases where two ducks are the same height, you wish to sort them by color (in the order of red, then orange, then yellow, then green, then blue, and then purple). For ducks which have the same height and color, you wish to sort them based on the type of hat that they have (first propeller hat, then party hat, then baseball cap, and finally fedora). Since ducks are so unique, you can rest assured that there does not exist two ducks which are identical in their height, color, and hat type.

Input

The first line will consist of a single integer, $1 \leq n \leq 10^6$, denoting the number of ducks to follow. The next n lines will each contain three single-space-separated values: the height of the duck $0 < h_i \leq 100$, where the height is a floating point value in centimeters; the color of the duck, $c_i \in \{$"Red","Orange","Yellow","Green","Blue","Purple"$\}$; and the type of hat that the duck is wearing, $t_i \in \{$"Propeller","Party","Baseball","Fedora"$\}$.

Output

Output the ducks in sorted order, using the same format as the input. That is, a duck is formatted as three single-space-separated values: the height of the duck, the color of the duck, and the type of hat that the duck is wearing

Example Input File

```
10
9.3 Yellow Propeller
1.002 Green Fedora
9.3 Red Fedora
10.7 Blue Party
30.03 Yellow Baseball
10.7 Orange Baseball
9.3 Red Baseball
33.3 Blue Party
60.8 Green Propeller
40.6 Purple Baseball
```

Example Output to Screen

```
1.002 Green Fedora
9.3 Red Baseball
9.3 Red Fedora
9.3 Yellow Propeller
10.7 Orange Baseball
10.7 Blue Party
30.03 Yellow Baseball
33.3 Blue Party
40.6 Purple Baseball
60.8 Green Propeller
```

10. Dots

Program Name: Dots.java **Input File: dots.dat**

Connects-the-Dots is a children's activity where, given various numbered dots on a piece of paper, take a pencil, place it on the dot numbered 1, draw a straight line to the dot numbered 2, draw a straight line to the dot numbered 3, and so on until all dots are connected. Typically, such an activity is intended to draw some intricate figure; however, being much older now, you no longer see the joy in this monotonous activity. Rather, you are interested in determining how much lead or ink gets wasted by drawing these lines in the order that the paper "suggests" you follow. However, for this to be an adequate comparison, you must first determine what set of straight lines between dots not only connects all points together, but also does so using the least amount of lead/ink. It should be noted that two dots, u and v, are considered to be "connected" if a series of one or more already drawn straight lines can be used to traverse from u to v without drawing any additional straight lines.

Input

The first line will consist of a single integer, $1 \leq n \leq 10^3$, denoting the number of test cases to follow. The next n testcases will each start with a single line which will consist of a single integer, $2 \leq D \leq 10^3$, denoting the number of dots to follow. The next D lines will consist of two space-separated integers, $-(2^{31}) - 1 \leq x_i, y_i \leq 2^{31} - 1$, denoting the coordinates of the i^{th} dot. Dots are given in increasing order according to the order that they should be drawn in the original Connect-the-Dots problem.

Output

For each of the n testcases, on its own line, print a single floating-point number expressed to 10 decimal points of precision, denoting the total amount of lead/ink that is wasted by following the Connect-the-Dots order.

Example Input File

```
2
7
-3 0
-1 2
1 2
3 0
1 -2
-1 -2
0 0
10
5 0
-40 5
61 3
50 9
9 4
50 78
-89 0
-87 50
60 -8
67 -50
```

Example Output to Screen

```
0.5923591472
368.0801362913
```

11. Gopher

Program Name: Gopher.java **Input File: gopher.dat**

You, and your pet gopher, Hazel, have recently landed yourself an internship at the illustrious company elgooG™. On your first day as an elgooG™ intern, you learned that you will be programming in elgooG™'s own proprietary language: Go. Appalled by the fact that you are no longer using Java, and jealous that the mascot of the Go programming language is another gopher, Hazel decides to march herself down to elgooG™'s headquarters where she is hoping that she can talk some sense into the creative team at elgooG™ into making her the new mascot for the Go programming language. While this may be a somewhat difficult journey for an ordinary gopher, Hazel has taken preemptive measures for such a journey by constructing underground tunnels ahead of time which she can use to help navigate the otherwise difficult areas on the surface. Moreover, being the engineer that she is, Hazel is interested in taking the shortest path from her owner's house to elgooG™'s headquarters, knowing that she can only move in the directions of north, south, east, and west.

Input

The first line will consist of a single integer, $1 \leq n \leq 200$, denoting the number of test cases to follow. The next n testcases will each start with a line consisting of two integers, $1 \leq R, C \leq 500$, denoting the number of rows and columns of the backyard, respectively. The next R lines will each consist of C valid characters denoting the layout of the surface, followed by R lines each consisting of C valid characters denoting the tunnels below. Valid characters consist of '`#`' representing unwalkable space, '`.`' representing walkable space, '`O`' representing a hole from the surface to the tunnels below (or vice versa), '`H`' representing the location of your home, and '`G`' representing the location of elgooG™'s headquarters. It should be noted that a hole appears at coordinate (r, c) on the surface if and only if a hole appears at coordinate (r, c) in the tunnels below, and such a hole can only be used to travel from coordinate (r, c) on the surface to coordinate (r, c) in the tunnels below (or vice versa). Additionally, it is guaranteed that both the location of your home and the elgooG™'s headquarters will be on the surface.

Output

For each of the n testcases, on its own line, either output the string "`Nowhere left to go-pher.`" or output a single integer denoting the shortest path from your home to elgooG™ headquarters. Note that it takes 1 unit of time to move from a walkable space to either another walkable space or a hole, but it does not take time to move through a hole.

Example Input File

```
3
4 6
H.O#OG
...###
####O#
#O...#
##O#O#
##.#.#
##.#O#
#O.###
1 11
```

Continues next column…

Continued from previous column…

```
H..O###O..G
###O.#.O###
4 10
O#O.O#....
.H#...#..G
##.#.#.O..
.O.O.#....
O.O#O.....
.########.
.####..O#.
.O#O..#...
```

Example Output to Screen

```
13
Nowhere left to go-pher.
16
```

12. Plagiarism

Program Name: Plagiarism.java **Input File: plagiarism.dat**

You and your friend are working together on an English homework assignment thinking that it was a group project, but suddenly you remember that this is individual work! Unfortunately, being the smarter person between you and your friend, you are left with the task of re-writing your copy of the assignment to be distinct enough from your friend's, so your teacher won't suspect you of cheating. Thankfully, your teacher doesn't use a particularly sophisticated plagiarism checker. Rather, the piece of software that your teacher uses operates on the longest common subsequence (LCS) problem. That is, it first determines the length of the LCS between two texts, divides that value into the total number of characters in the text being tested, and represents that value as a percentage. It should be noted that a subsequence is a sequence of elements that appear in the same order, but not necessarily consecutively. Additionally, if text A is being compared to text B, then the length of text A should be divided into the length of the LCS. Given two pieces of text, determine the plagiarism score between the two.

Input

The first line will consist of a single integer, $1 \leq n \leq 100$, denoting the number of testcases to follow. The next n testcases will consist of two lines of text denoting your and your friend's copy of the assignment (text A and B, respectively). Each line of text will be between 1 and 10^3 characters in length. It should be noted that whitespace is considered towards the line length, as well as the plagiarism score, and capital letters are unique from their lowercase variants.

Output

For each testcase, on its own line, print the string "<DESC>: <SCORE>%" where <DESC> is a text description of the plagiarism score, and <SCORE> is the plagiarism score, represented as a percentage, expressed to 5 decimal places of precision. If the calculated plagiarism score is below 10%, then use the description "No significant plagiarism detected". If the score is below 33%, then use the description "Minimal plagiarism detected". If the score is below 67%, then use the description "Considerable plagiarism detected". Otherwise, use the description "Texts are nearly identical or identical".

Example Input File *(indented lines are continuation of previous line)*

```
2
Did you know that only sixty-four characters can fit on one line
As you can guess, the line above is sixty-four characters long
What does it mean to be a programmer? It's all about being a
     problem solver. Without that, knowing the syntax of a
     language is completely useless. In Conclusion, being a
     programmer isn't about knowing lots of algorithms, but more
     to do with being able to break down and solve problems.
What doth it mean to be a programmer? 'Tis all about being a
     solver of problems. Without such skill, knowing the syntax
     of a language is utterly in vain. Thus, being a programmer
     is not about possessing a myriad of algorithms, but rather
     the ability to dissect and resolve the dilemmas at hand.
```

Example Output to Screen

```
Considerable plagiarism detected: 56.25000%
Texts are nearly identical or identical: 74.03509%
```